Mateescu Corina-Elena CEN3.2A
Project 25

PROJECT 1 - PIC24

Instructions for project 25:

Project 25

Non-jump Instructions	Flags	Jump Instructions
BTG Ws,#bit4	OV	BRA OV,Expr
INC Ws,Wd	C	BRA C,Expr
CLR Wd	N	BRA N,Expr
SUBR Wb,#lit5,Wd	Z	BRA Z,Expr

Opcode table:

Encoding	2222 3210	1111 9876	1111 5432	11 1098	7654	3210	Flags
ADD Wb, Ws, Wd	0100	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
SUB Wb, Ws, Wd	0101	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
AND Wb, Ws, Wd	0110	0www	wBqq	qddd	dppp	ssss	N, -, Z, -
IOR Wb, Ws, Wd	0111	0www	wBqq	qddd	dppp	ssss	N, -, Z, -
MOV f, Wnd	1000	0fff	ffff	ffff	ffff	dddd	none
MOV Wns, f	1000	1fff	ffff	ffff	ffff	ssss	none
BRA Expr	0011	0111	nnnn	nnnn	nnnn	nnnn	none
BTG Ws, #bit4	1010	0010	bbbb	0B00	0ppp	ssss	none
INC Ws, Wd	1110	1000	0Bqq	qddd	dppp	ssss	N, OV, Z, C
CLR Wd	1110	1011	0Bqq	qddd	d000	0000	none
SUBR Wb, #lit5, Wd	0001	0www	wBqq	qddd	d11k	kkkk	N, OV, Z, C
BRA OV, Expr	0011	0000	nnnn	nnnn	nnnn	nnnn	none
BRA C, Expr	0011	0001	nnnn	nnnn	nnnn	nnnn	none
BRA N, Expr	0011	0011	nnnn	nnnn	nnnn	nnnn	none
BRA Z, Expr	0011	0010	nnnn	nnnn	nnnn	nnnn	none

*for BTG, Ws is both source register and destination register, so a mux is needed for WD, because it is on bits 10-7, and on bits 3-0

Mateescu Corina-Elena
Project 25

Flags table:

Encoding	**Flag N (CE_NF)**	**Flag OV (CE_OVF)**	**Flag Z (CE_ZF)**	**Flag C (CE_CF)**	**Flags**
ADD Wb, Ws, Wd	1	1	1	1	N, OV, Z, C
SUB Wb, Ws, Wd	1	1	1	1	N, OV, Z, C
AND Wb, Ws, Wd	1	0	1	0	N, -, Z, -
IOR Wb, Ws, Wd	1	0	1	0	N, -, Z, -
MOV f, Wnd	0	0	0	0	none
MOV Wns, f	0	0	0	0	none
BRA Expr	0	0	0	0	none
BTG Ws, #bit4	0	0	0	0	none
INC Ws, Wd	1	1	1	1	N, OV, Z, C
CLR Wd	0	0	0	0	none
SUBR Wb, #lit5, Wd	1	1	1	1	N, OV, Z, C
BRA OV, Expr	0	0	0	0	none
BRA C, Expr	0	0	0	0	none
BRA N, Expr	0	0	0	0	none
BRA Z, Expr	0	0	0	0	none

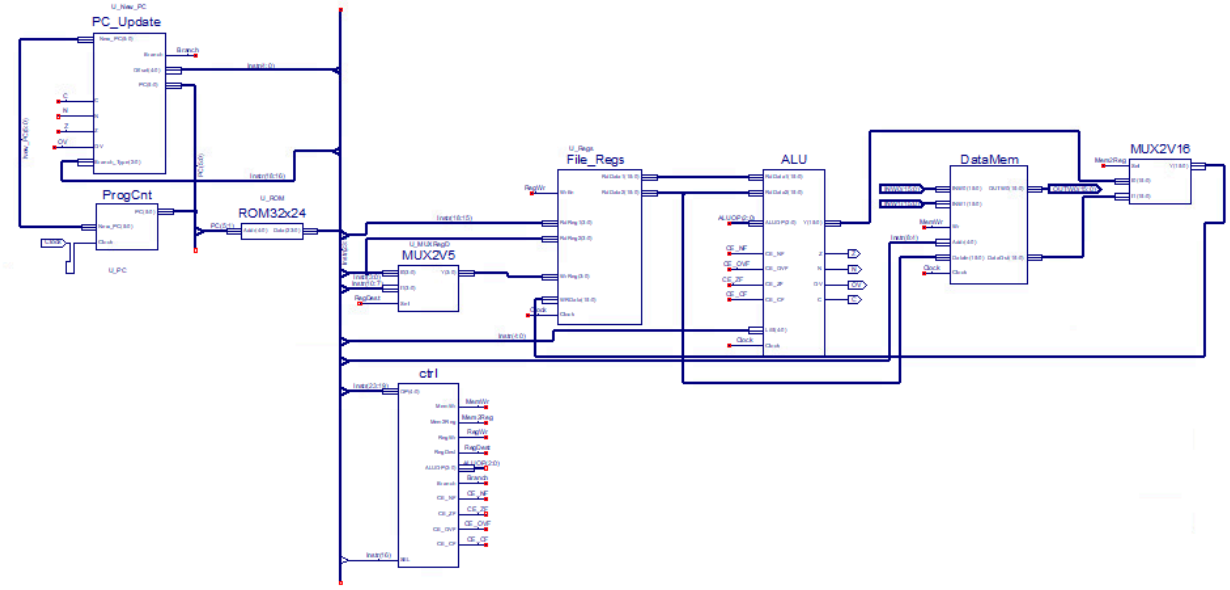
Mateescu Corina-Elena CEN3.2A
Project 25

Control block table:

	OP	SEL	CE_NF	ALUOP	MemWr	Mem2Reg	RegWr	RegDest	CE_OVF	CE_ZF	CE_CF	Branch
ADD Wb, Ws, Wd	01000	x	1	000	0	0	1	1	1	1	1	0
SUB Wb, Ws, Wd	01010	x	1	001	0	0	1	1	1	1	1	0
AND Wb, Ws, Wd	01100	x	1	010	0	0	1	1	0	1	0	0
IOR Wb, Ws, Wd	01110	x	1	011	0	0	1	1	0	1	0	0
MOV f, Wnd	10000	x	0	xxx	0	1	1	0	0	0	0	0
MOV Wns, f	10001	x	0	xxx	1	x	x	x	0	0	0	0
BRA Expr	00110	x	0	xxx	0	x	0	x	0	0	0	1
BTG Ws, #bit4	10100	x	0	100	0	0	1	0	0	0	0	0
INC Ws, Wd	11101	0	1	101	0	0	1	1	1	1	1	0
CLR Wd	11101	1	0	110	0	0	1	1	0	0	0	0
SUBR Wb, #lit5, Wd	00010	x	1	111	0	0	1	1	1	1	1	0

BRA OV, Expr	0011 0	x	0	xxx	0	x	0	x	0	0	0	1
BRA C, Expr	0011 0	x	0	xxx	0	x	0	x	0	0	0	1
BRA N, Expr	0011 0	x	0	xxx	0	x	0	x	0	0	0	1
BRA Z, Expr	0011 0	x	0	xxx	0	x	0	x	0	0	0	1

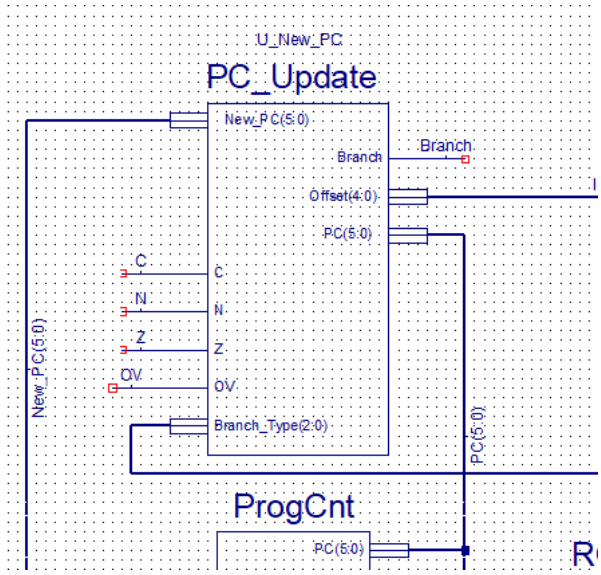
Block diagram of the processor that executes both common and specific instructions:



Mateescu Corina-Elena CEN3.2A
Project 25

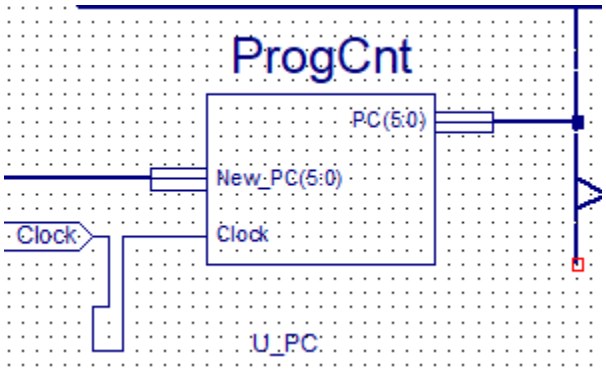
PIC24 blocks:

PC_Update



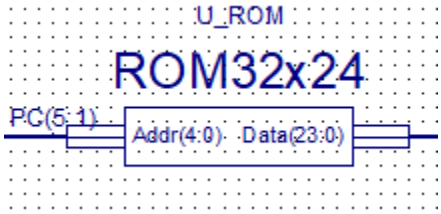
This module calculates the next **Program Counter** (**PC**) value. By default, the **PC** is incremented by 2 for sequential instruction execution. If the **Branch** signal is asserted and the condition specified by **Branch_Type** matches the processor status flags (**OV, C, N, Z**), the module adds the offset to the incremented **PC** to perform a branch. Otherwise, the **PC** continues sequentially.

ProgCnt



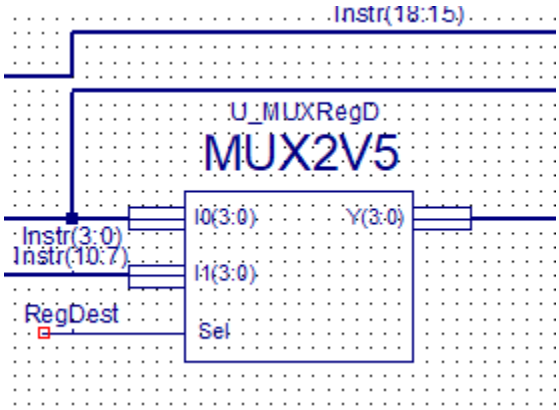
The **ProgCnt** module implements the Program Counter register. On each rising edge of the clock, it loads the value of **New_PC** into **PC**, updating the current instruction address. The **PC** is initialized to “`000000`” at startup.

ROM32x24



This module implements a read-only memory (**ROM**) used to store test programs and instruction sequences. The **ROM** outputs a 24-bit instruction corresponding to the provided address. It is used to supply predefined instruction patterns for validating processor functionality and verifying the execution of different instructions and control behaviors.

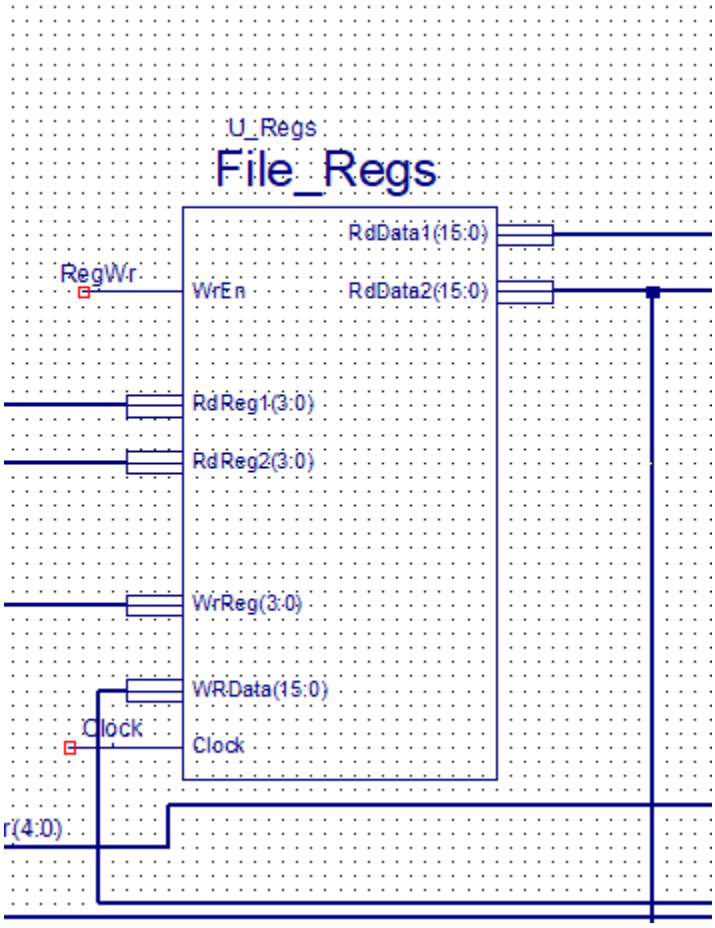
MuxRegD



This module implements a 2-to-1 multiplexer for 4-bit register data inputs. Controlled by the **Sel** signal, it selects either **I0** or **I1** and forwards the chosen value to the output **Y**. It is used to control data selection within the register datapath.

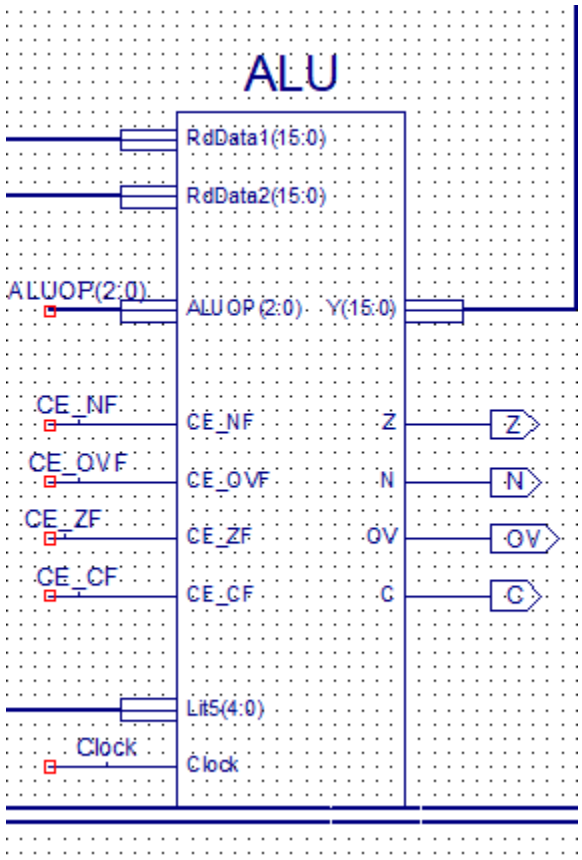
Mateescu Corina-Elena CEN3.2A
Project 25

File_Regs



This module implements a 16×16-bit register file with two read ports and one write port. On the rising edge of the **Clock**, if **WrEn** is asserted, the **WRData** is written into the register specified by **WrReg**. The registers specified by **RdReg1** and **RdReg2** are continuously output on **RdData1** and **RdData2**, allowing simultaneous read operations. It is used to store and provide operand data within the processor datapath.

ALU



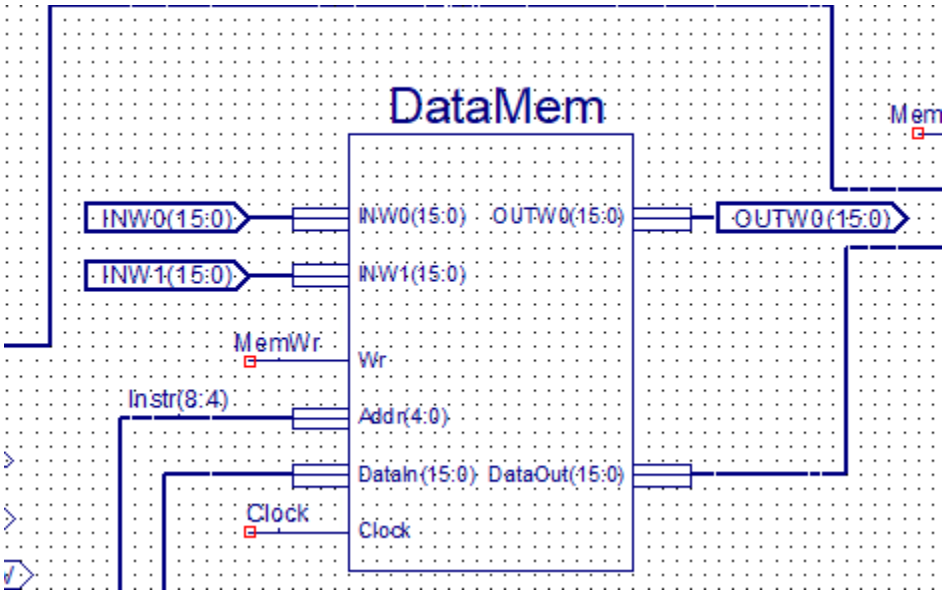
The **ALU** performs arithmetic and logic operations on 16-bit operands (**RdData1** and **RdData2**) based on the **ALUOP** control signal. Supported operations include addition, subtraction, increment, logical AND/OR, bit manipulation, and subtraction with literal (**SUBR**).

Outputs and Flags:

- **Y**: Result of the selected operation.
- **Z**: Zero flag, set when the result is zero.
- **N**: Negative flag, reflecting the sign of the result.
- **OV**: Overflow flag, set when arithmetic overflow occurs.
- **C**: Carry flag, set for operations that generate a carry out.

Clock Enable Signals:
The flags (**Z, N, OV, C**) are updated on the rising edge of **Clock** only when their corresponding enable signals (**CE_ZF, CE_NF, CE_OVF, CE_CF**) are active.

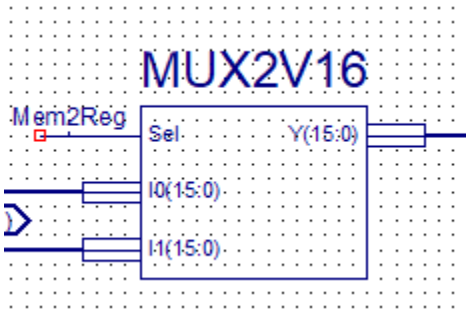
DataMem



The **DataMem** module implements a 16×16-bit memory with separate input and output ports. It supports synchronous writes when **Wr** is asserted and the address indicates internal memory, while reads return either internal memory data or external inputs (**INW0**, **INW1**) based on the address. The **OUTW0** port captures written data under specific address conditions. It is used for storing and retrieving operand and result data within the processor.

Mateescu Corina-Elena CEN3.2A
Project 25

MUX2v16



The **MUX2V16** is a 16-bit 2-to-1 multiplexer responsible for selecting the data source for register write operations.

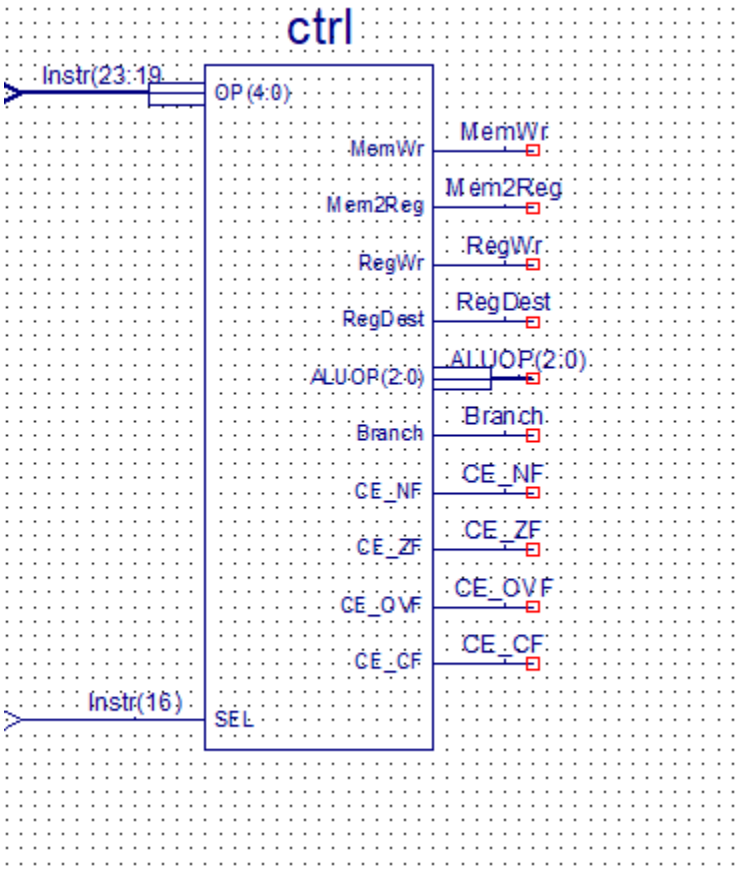
Source Selection:
Based on the **Sel** (Mem2Reg) control signal, the multiplexer chooses which input is forwarded to the destination register.

Inputs (I0 and I1):

- **I0:** Receives the processed result from the ALU.
- **I1:** Receives data read from the Data Memory (**DataMem**).

Output (Y):
The output **Y** contains the 16-bit word from the selected input and is connected to the write-back port of the Register File.

ctrl



The **ctrl** module generates control signals for the processor based on the 5-bit opcode (**OP**) and the **SEL** input. It determines the operation of the ALU, register file, memory, branching, and status flag updates.

Functions:

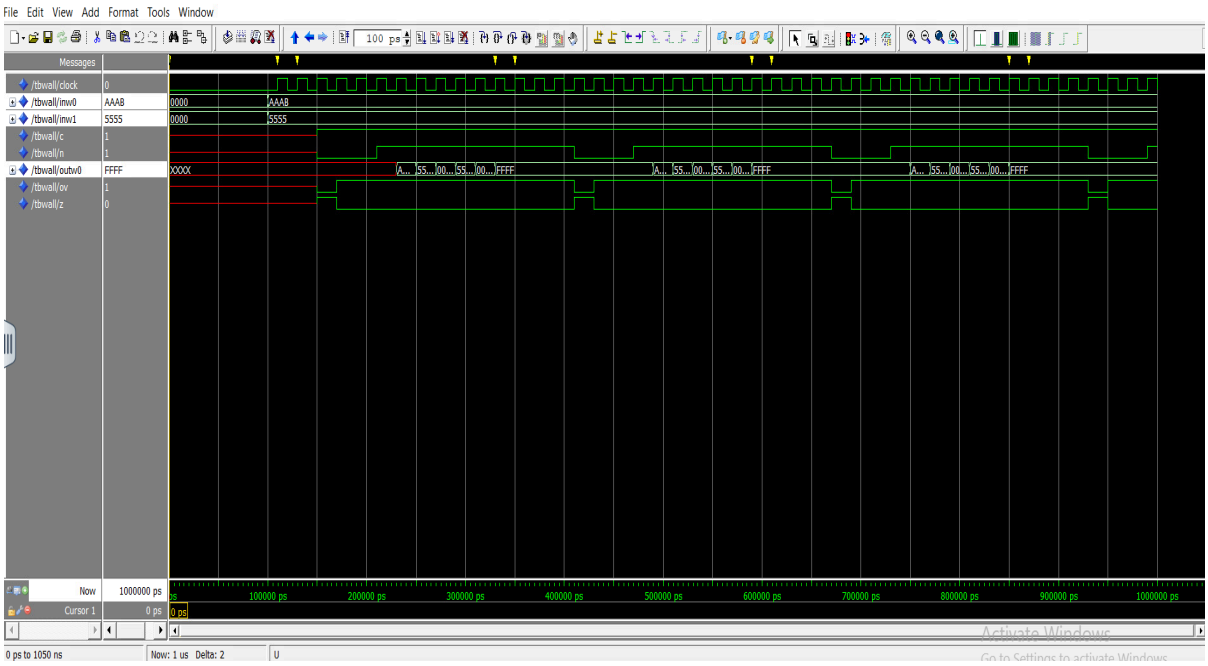
- **ALUOP:** Selects the ALU operation according to the instruction.
- **RegDest:** Chooses the destination register for write operations.
- **RegWr:** Enables writing to the register file.
- **MemWr / Mem2Reg:** Controls memory writes and selecting memory data for register writes.
- **Branch:** Indicates whether a branch instruction should be executed.

- **CE_NF, CE_ZF, CE_OVF, CE_CF:** Enable updating of the ALU status flags (Negative, Zero, Overflow, Carry) depending on the instruction type.

This unit ensures proper coordination of the processor datapath by asserting signals required for each instruction type.

Simulation:

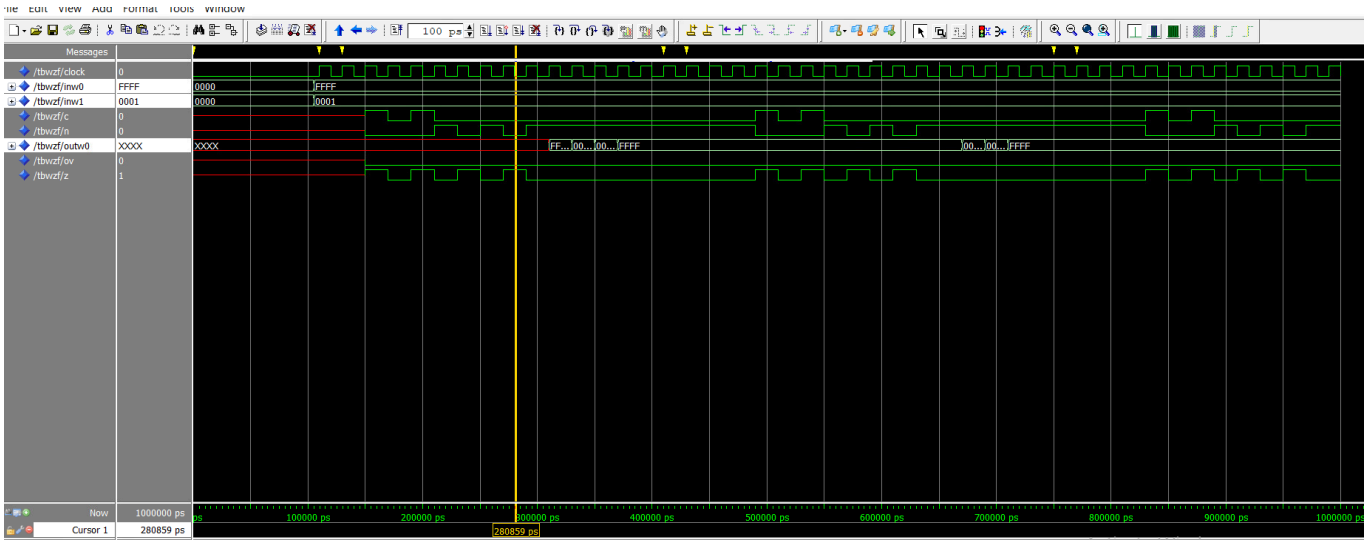
All



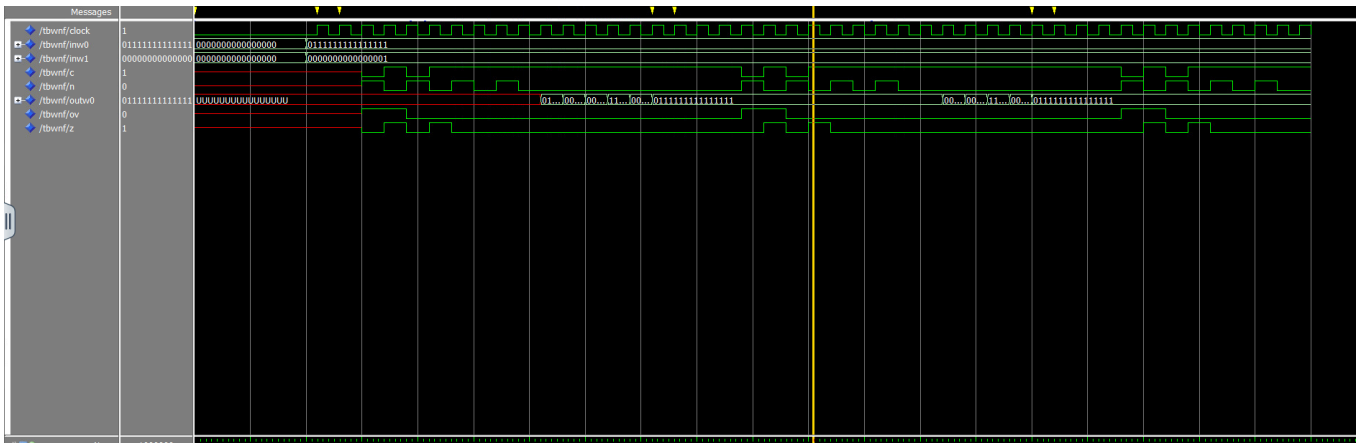
Mateescu Corina-Elena CEN3.2A

Project 25

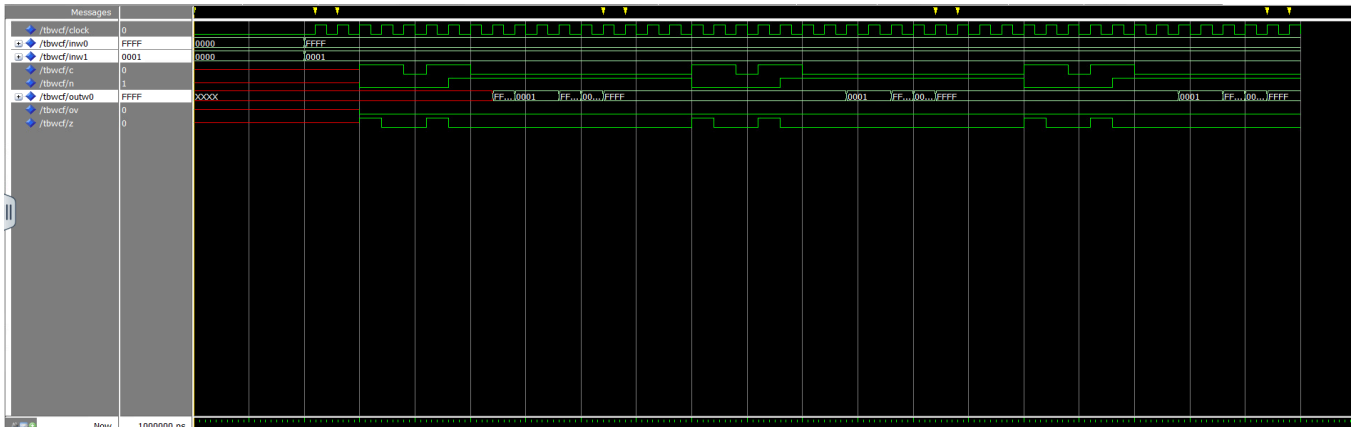
Z flag



N flag



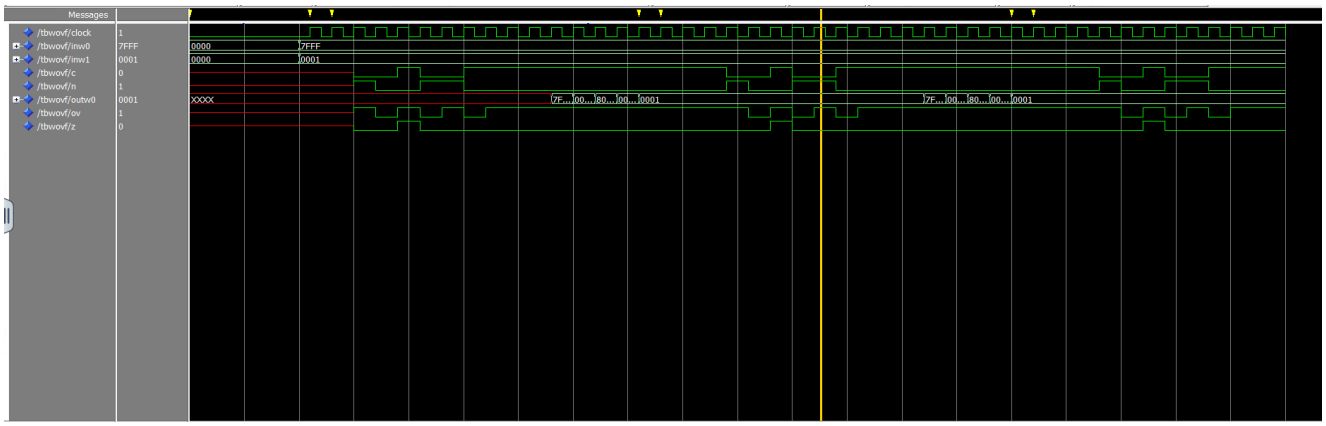
C flag



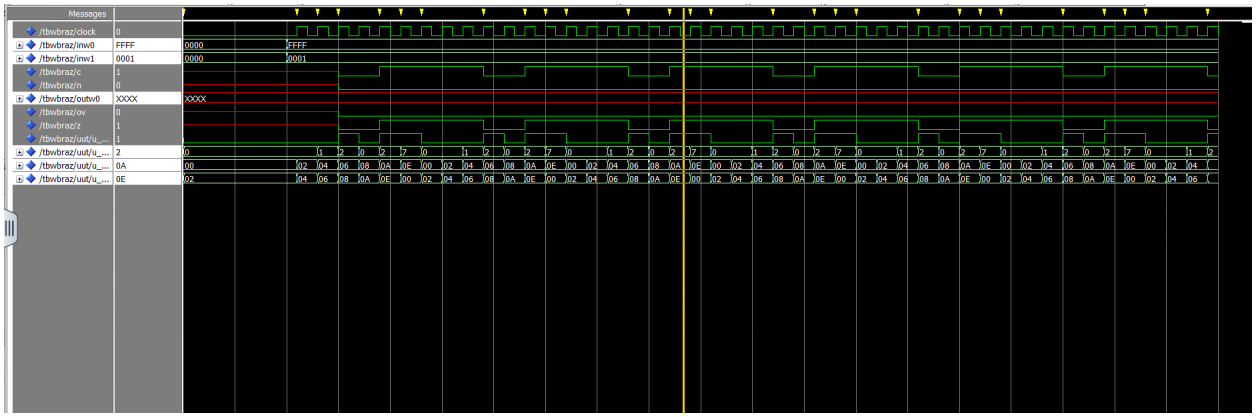
Mateescu Corina-Elena CEN3.2A

Project 25

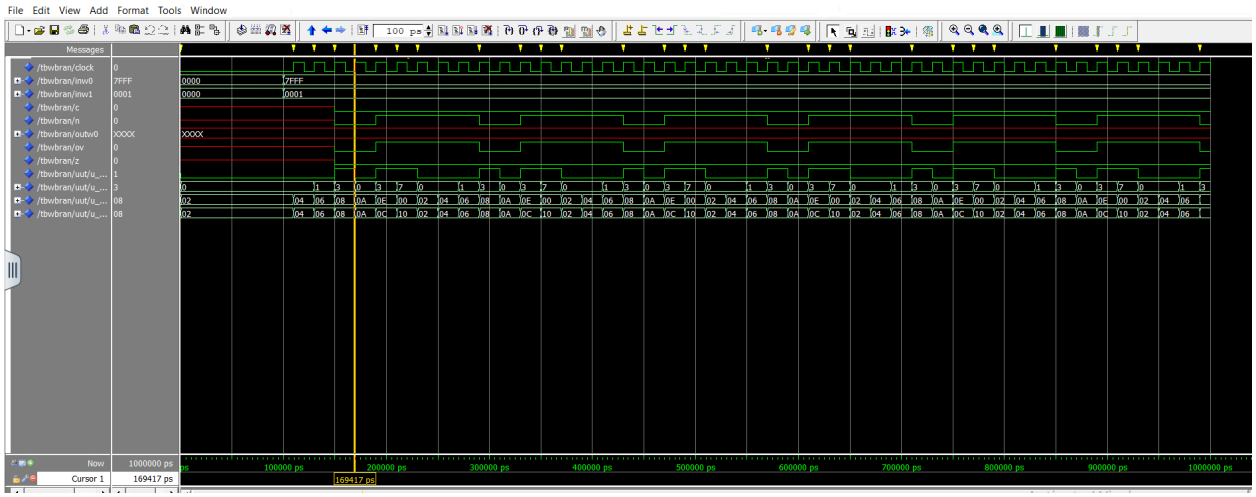
OVF



Branch Z



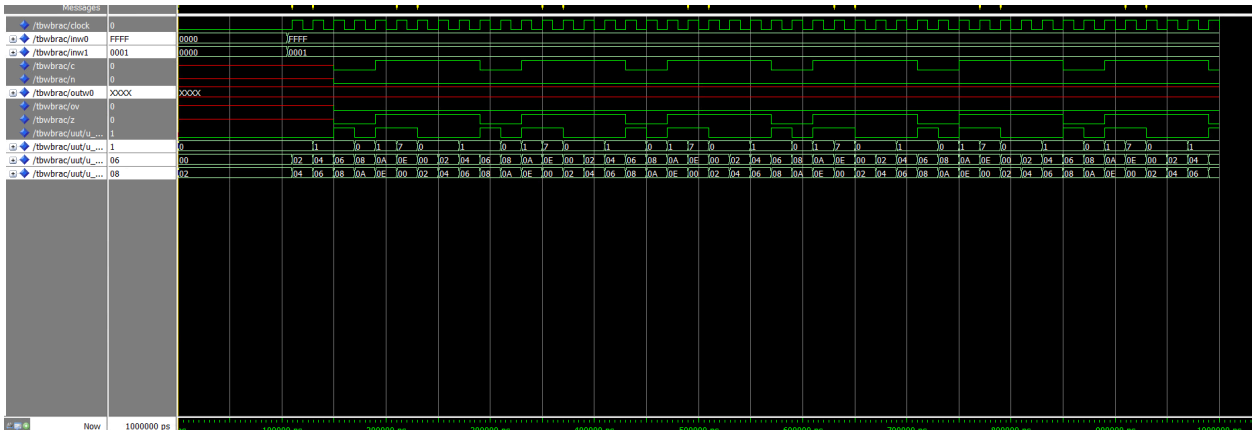
Branch N



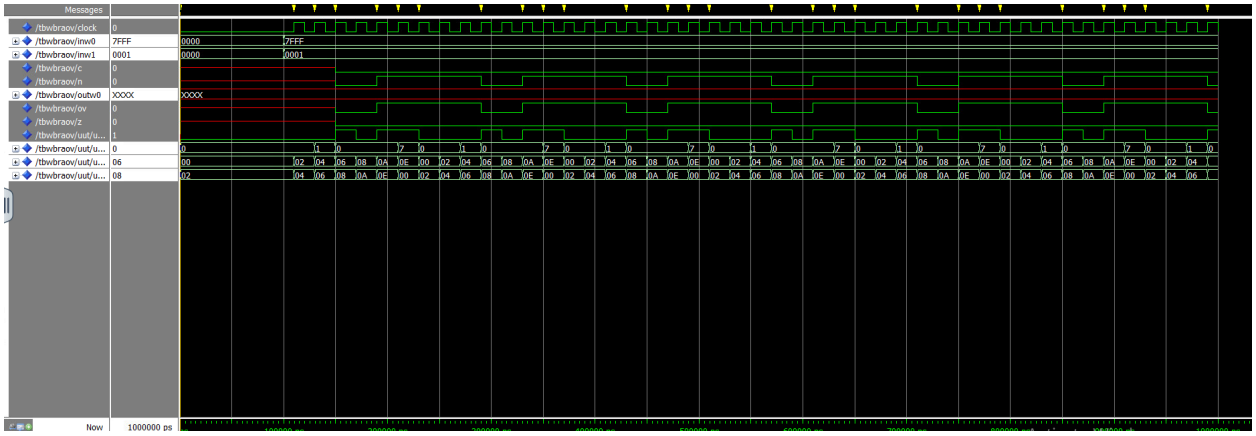
Mateescu Corina-Elena CEN3.2A

Project 25

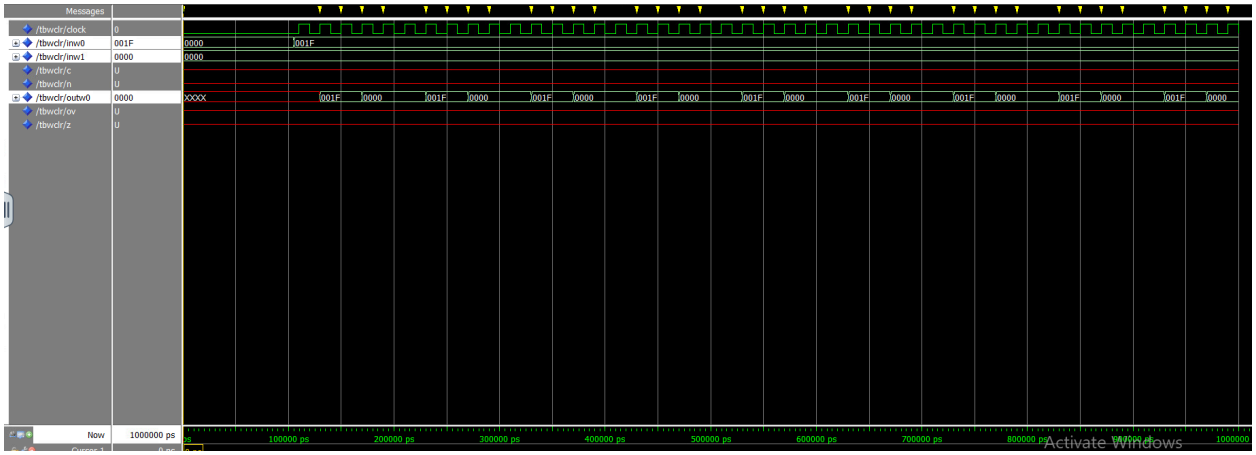
Branch C



Branch OVF



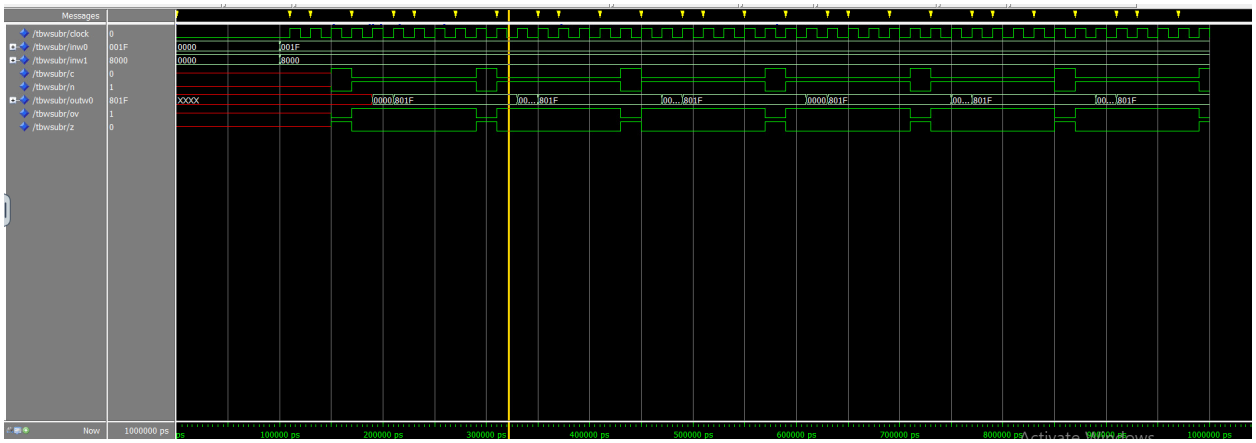
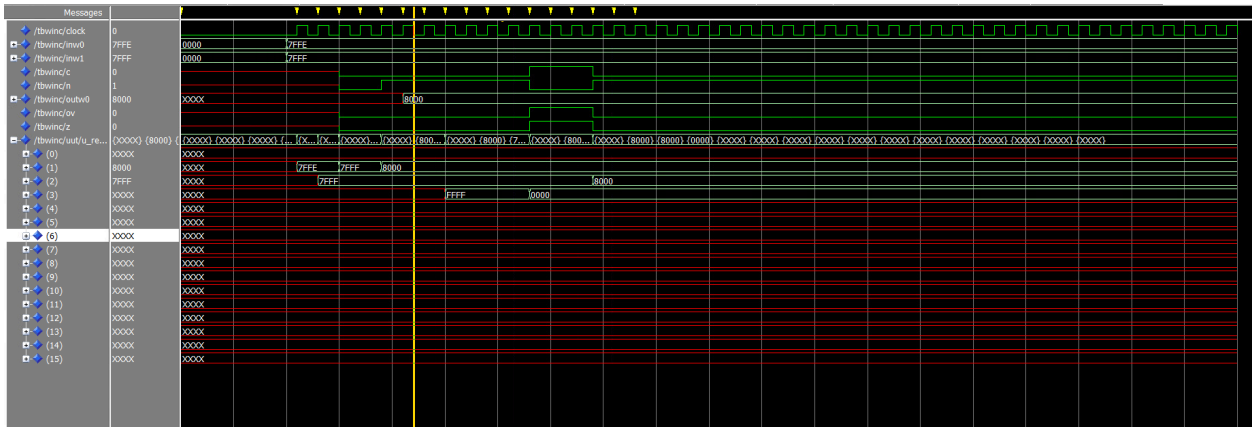
CLR Wd



Mateescu Corina-Elena CEN3.2A

Project 25

SUBR Wb, #lit5, Wd



INC Ws, Wd